

CSSE490 - Lab 2: Policy Iteration and Value Iteration

Kevin Cotellesso

3-20-2026

In this lab, we will implement policy iteration and value iteration to solve the Frozen lake problem.

Let's first get ourselves familiarized with the environment.

```
In [2]: import numpy as np
import gymnasium as gym

env = gym.make('FrozenLake-v1', map_name="4x4", is_slippery=True) # or you
env.reset()
n_state = env.unwrapped.observation_space.n
n_action = env.unwrapped.action_space.n
print("# of actions", n_action)
print("# of states", n_state)
P = env.unwrapped.P
# At state 14 apply action 2
for prob, next_state, reward, done in P[14][2]:
    print("If apply action 2 under state 14, there is %3.2g probability it w
        prob, next_state, reward))
```

of actions 4

of states 16

If apply action 2 under state 14, there is 0.33 probability it will transiti
on to state 14 and yield reward as 0

If apply action 2 under state 14, there is 0.33 probability it will transiti
on to state 15 and yield reward as 1

If apply action 2 under state 14, there is 0.33 probability it will transiti
on to state 10 and yield reward as 0

First, let's write utility functions we will use to run experiments.

Given certain policy, how can we compute the value function for each state.

```
In [3]: def compute_policy_v(env, policy, gamma=1.0):
    # The goal of this function is to calculate the Q(s,a) for each action u
    eps = 1e-10
    V = np.zeros(n_state)
    while True:
        prev_V = np.copy(V)
        for state in range(n_state):
            action = policy[state] # Deterministic!
            V[state] = 0
            for prob, next_state, reward, done in P[state][action]:
                V[state] += prob * (reward + gamma * prev_V[next_state])
```

```

        # V is sum of all Q(s,a) weighted by probability of taking t
        # In our case, after one iteration, our policy is deterministic
        # include the Q from the next_state.
    if (np.sum((np.fabs(prev_V - V))) <= eps):
        break
    return V

```

Let's test a random policy

```

In [4]: random_policy = [np.random.choice(n_action) for _ in range(n_state)]
print(random_policy)
rand_v = compute_policy_v(env, random_policy)
print(np.round(rand_v, 5))

[1, 3, 0, 0, 0, 3, 3, 1, 2, 1, 0, 0, 1, 1, 1, 2]
[0.07384 0.05274 0.03165 0.01582 0.09494 0.         0.01055 0.         0.11603
 0.25316 0.31013 0.         0.         0.33333 0.66667 0.         ]

```

Given value function, we need to extract the best policy from it.

```

In [5]: # need a Q here, since we are getting the best policy by argmaxing
def extract_policy(V, gamma=1.0):
    policy = np.zeros(n_state) # Action for each state
    Q = np.zeros((n_state, n_action))
    for state in range(n_state):
        for action in range(n_action):
            for prob, next_state, reward, done in P[state][action]:
                Q[state, action] += prob * (reward + gamma*V[next_state])
            policy[state] = np.argmax(Q[state]) # Best action for each state
    return policy

```

```

In [6]: print(extract_policy(rand_v))

[0. 3. 3. 3. 0. 0. 0. 0. 3. 1. 0. 0. 0. 2. 1. 0.]

```

Now let's start with a random policy and compute the value then extract new policy. Do this recursively will improve the policy

```

In [7]: def run_episode(env, policy, gamma=1.0, render=False):
    """ Runs an episode and return the total reward """
    obs, _ = env.reset()
    total_reward = 0
    step_idx = 0
    while True:
        if render:
            env.render()
        obs, reward, truncated, done, _ = env.step(int(policy[obs]))
        # this will calculate the return for the first step
        total_reward += (gamma ** step_idx * reward)
        step_idx += 1
        if done or truncated:
            break
    return total_reward

```

```

In [8]: # Policy iteration:

```

```

# 1. Given a deterministic policy (state -> action), compute value of every
# 2. Given these values, get a new policy.
# 3. Run an episode to get a score
# Repeat until policy does not change.
policy_PI = random_policy
max_iterations = 200000
gamma = 1.0
for i in range(max_iterations):
    policy_v = compute_policy_v(env, policy_PI)
    new_policy = extract_policy(policy_v)
    score = run_episode(env, new_policy, gamma, False)
    print("iteration %i: score %f" % (i, score))
    if (np.all(policy_PI == new_policy)):
        print('Policy-Iteration converged at step %d.' % (i+1))
        break
    policy_PI = new_policy

scores = [run_episode(env, policy_PI, gamma, False) for _ in range(1000)]
print("PI Final score:", np.mean(scores))

```

```

iteration 0: score 1.000000
iteration 1: score 1.000000
Policy-Iteration converged at step 2.
PI Final score: 0.74

```

Part 2: Value Iteration

```

In [9]: # Value iteration: Converge V then find Q then find policy.
policy_VI = random_policy
max_iterations = 2000
gamma = 1.0
eps = 1e-10
Vs = np.zeros((max_iterations, n_state))
for iter in range(1, max_iterations):
    # 1. Loop through each state.
    Q = np.zeros((n_state, n_action)) # Reset this between iterations
    for state in range(n_state):
        # Get the Q-function for each action
        for action in range(n_action):
            for prob, next_state, reward, done in P[state][action]:
                Q[state, action] += prob * (reward + gamma*Vs[iter-1][next_s
            Vs[iter][state] = np.max(Q[state]) # The next iteration of this stat
    # Check to see if we've converged
    if(np.sum(np.abs(Vs[iter]-Vs[iter-1])) < eps):
        print(f"Value matrix converged after {iter} iterations!")
        break

V = Vs[iter] # Why is Python mad about this??
assert all(abs(v - 0.82352941) < 1e-5 for v in V[0:5]), "Value function is n
assert all(abs(v - 0) < 1e-5 for v in V[[5,7,11,12,15]]), "Value function is

# Get the policy from this value matrix.
# Actually, Q should be up to date, so we can simply pick the best action fr
for state in range(n_state):

```

```
policy_VI[state] = np.argmax(Q[state]) # Best action for this state

# Run it a couple times
scores = [run_episode(env, policy_VI, gamma, False) for _ in range(1000)]
print("VI Final score:", np.mean(scores))
```

Value matrix converged after 877 iterations!
VI Final score: 0.757

Part 3: Questions

1. Policy iteration needs an inner loop to compute the value for a fixed policy. This inner loop does the same thing as the outer loop for value iteration: for each state/action pair, find the Q value of that state action pair. We need this inner loop in order to converge the value function.
2. Policy iteration takes 6 outer iterations. Value iteration takes 877 iterations to converge. These numbers are misleading, however, since at each of policy iteration's six steps it had to run the inner loop many times to converge the value function for that policy.
3. The Bellman optimality equation is greedy, calculating the Q function for all actions and always taking the highest valued action. We get a similar effect with policy iteration with the Bellman expectation equation, since the expected value of a policy that chooses the highest value IS the max (over a) of the Q function at that state.
4. Average score is from 0.72 to 0.8. This is because there is a region of FrozenLake where it is always possible to fall in a hole, no matter how careful you are.
5. Value iteration seems to be a lot faster for this problem, which has a very limited action space. However, if the action space were a lot larger, it would be difficult to generate Q-values for every state-action pair. In this case, a policy iteration method, which estimates the value of each state using an existing policy, would be less compute intensive.